

SinkAware: Approximate Low-Rank Fixed-Sink Priors for Attention

Anonymous authors

Paper under double-blind review

Abstract

Attention sinks are fixed early tokens that often receive substantial attention. A tempting systems idea is to precompute or reuse their contribution. We show that exact static reuse is invalid under standard softmax attention because fixed sink-token logits remain query-dependent. We therefore test a narrower approximation: per-head low-rank predictors for fixed sink logits while keeping all non-sink scores exact. On Mac-local distilgpt2 traces, rank-2 prediction reduces held-out attention-output relative L2 error to 0.141 versus 0.170 for a position-only predictor, while the cost model estimates 0.531x exact four-sink QK multiply-adds. We also add a CPU reference for the exact operator a future kernel must implement: replace only sink logits, keep tail logits exact, and verify exact sink logits reproduce exact attention. This is not an end-to-end quality or GPU speed result. It is a bounded native-GPU gate for deciding whether approximate sink prediction is worth implementing.

1 Problem

For query q_t , sink keys K_s , and non-sink keys K_n , the sink logits $q_t K_s^\top$ are still query-dependent. Any exact method that skips them changes the softmax denominator or attention weights. SinkAware kills that exact static-prior branch and revives only an approximate one.

2 Approximate Branch

The live branch predicts fixed sink-token logits from a low-dimensional query feature. Non-sink logits remain exact. The branch is useful only if the approximation is cheap enough and attention-output drift stays bounded.

Predictor	Sink RMSE	Sink-mass MAE	Attention L1	Output rel-L2
static	1229.509	0.095	0.217	0.193
position	1197.390	0.076	0.175	0.170
rank1	1079.541	0.067	0.157	0.160
rank2	1001.506	0.055	0.135	0.141
rank4	838.320	0.045	0.115	0.122
rank8	699.232	0.040	0.104	0.107

Table 1: 48-trace distilgpt2 softmax/output probe. Lower is better.

3 Reference and Kernel Gates

The Phase 3 reference specifies the exact computation to preserve during kernel work. Given query q , keys K , values V , and predicted fixed-sink logits $\hat{s}$, SinkAware computes all tail logits $q K_{tail}^\top$ exactly, replaces only the first sink-token logits with $\hat{s}$, and then runs the ordinary softmax denominator over the concatenated vector. Unit tests verify that when

$\hat{s}$ equals exact sink logits, the reference reproduces exact attention to numerical tolerance. This keeps the live branch separate from the killed exact-static-reuse branch.

The Phase 4 Triton-interpreter scaffold now targets the same operator for a one-head scalar-output synthetic gate. In the current Mac environment triton is not importable in `venv_arm64`, so those tests are written but skip locally. This is still useful for human review because the kernel correctness contract is explicit before any native timing work.

4 Decision

SinkAware is alive only as an approximate low-rank native GPU gate. Exact static reuse remains dead. Promotion requires rank-2 to preserve output quality while improving speed or memory traffic by at least 3% over exact attention on repeated native runs, and to beat the position-only predictor on quality.

5 Artifacts

- `experimental/sinkaware/phase2/real_qk_sink_softmax_output_probe.md`
- `experimental/sinkaware/phase2/qk_sink_cost_model.md`
- `experimental/sinkaware/phase3/approx_sink_attention_reference.md`
- `experimental/sinkaware/phase4/approx_sink_attention_triton_gate.md`
- `experimental/sinkaware/phase2/gpu_gate_runbook.md`
- `experimental/sinkaware/paper/reviewer_pack.md`